

## PORTFOLIO

# 컨테이너 오케스트레이션 & 통합 관제 플랫폼

여러 물리 서버에 흩어져 실행되는 다수의 독립 업무 컨테이너를, 하나의 웹 콘솔에서 생성·기동·모니터링·알림까지 통합 관리할 수 있게 만든 백엔드/인프라 프로젝트입니다.

입사서류 제출용 · 실제 사내 정보(IP·ID·업체명)는 전부 익명화했습니다



# 컨테이너 오케스트레이션 & 통합 관제 플랫폼

## 한 줄 요약

여러 물리 서버에 흩어져 실행되는 다수의 독립 업무 컨테이너를, 하나의 웹 콘솔에서 생성·기동·모니터링·알림까지 통합 관리할 수 있게 만든 백엔드/인프라 프로젝트입니다. Spring Boot 기반 3-tier 관제 시스템, TCP 소켓 기반 커스텀 파일 릴레이 시스템, Prometheus/Grafana 모니터링 확장, 카카오톡 알림 연동을 처음부터 설계·구현했습니다.

## 실행 화면

개요

이력

JobRadar

TMSManager

TMS통합 모니터링

Registry

Advisor

NodeExporter

MariaDB

TMS 컨테이너

기타 컨테이너

컨테이너 리스트

+ 신규 등록

| 삭제 | TMS_ID     | IMAGE_VERSION | MODE | SIZE | PORT_1 | PORT_2 | SERVER_IP    | STATUS | 관리 |
|----|------------|---------------|------|------|--------|--------|--------------|--------|----|
|    | 1000000001 | latest        | DEV  | 1    | 20001  | 30001  | 203.0.113.10 | 실행중    | 수정 |
|    | 1000000002 | latest        | DEV  | 1    | 20002  | 30002  | 203.0.113.10 | 실행중    | 수정 |
|    | 1000000003 | latest        | DEV  | 1    | 20003  | 30003  | 203.0.113.10 | 실행중    | 수정 |

시뮬레이션 테스트 (실행중인 TMS만 표시)

| TMS_ID     | 단말기 ID    | 송신파일개수 | 송신간격(초) | SND파일생성 | EAI_SERVER 도착여부       | RCV로 파일송신 (파일을 여기로 드래그)   |
|------------|-----------|--------|---------|---------|-----------------------|---------------------------|
| 1000000001 | 196510005 | 5      | 1       | 시작      | 6건 도착 <span>확인</span> | <div>여기에 파일을 드래그하세요</div> |
| 1000000002 | 196510005 | 5      | 1       | 시작      | 0건 도착 <span>확인</span> | <div>여기에 파일을 드래그하세요</div> |
| 1000000003 | 196510005 | 5      | 1       | 시작      | 0건 도착 <span>확인</span> | <div>여기에 파일을 드래그하세요</div> |

※ 서버 IP·컨테이너 ID는 예시 값으로 치환했습니다. 실제 화면 레이아웃/컴포넌트는 그대로입니다.

## 배경 및 문제 정의

- 업무 컨테이너 인스턴스가 여러 물리 서버에 분산 배치되어 있어, 서버별로 SSH 접속해서 `docker-compose up/down` 을 수동으로 실행해야 했고, 상태를 한눈에 파악할 방법이 없었음
- 인스턴스 간 파일 연동(발신/수신)이 필요한데, 기존 방식은 검증되지 않은 임시 스크립트에 의존

- 인프라 모니터링(CPU/메모리/컨테이너 상태)은 있었지만, 업무 지표(처리 건수 등)는 별도로 확인할 방법이 없었음
- 장애(컨테이너 다운) 발생 시 담당자가 화면을 계속 보고 있어야만 알 수 있었음

## 시스템 아키텍처

### 계층 구조와 설계 의도

| 계층                           | 역할                                           | 왜 이렇게 나뉘나                                                                             |
|------------------------------|----------------------------------------------|---------------------------------------------------------------------------------------|
| Manager                      | 웹 UI, 사용자 조작 진입점                             | 화면/사용성 관심사를 인프라 제어 로직과 분리                                                             |
| Server                       | 인증(API 키), 요청 라우팅, 상태 캐싱                     | Manager가 어떤 물리 서버에 어떤 인스턴스가 있는지 몰라도 되게 함 — 인스턴스가 나중에 다른 서버로 이전되어도 Manager 쪽 코드 변경 불필요 |
| Agent                        | 실제 <code>docker-compose</code> 실행, 파일 시스템 조작 | 물리 서버마다 1개씩 상주. 인증된 요청만 실제 명령을 실행                                                     |
| File Relay (Sender/Receiver) | 인스턴스별 파일 발신/수신 중계                            | HTTP가 아닌 자체 TCP 바이너리 프로토콜로 구현 — 대용량 파일 스트리밍과 pull 기반 수신을 모두 지원해야 했기 때문                |

### 핵심 성과

- 가동/중지, 신규 등록, 삭제를 웹 UI에서 원클릭으로 처리 — 서버별 SSH 접속 불필요
- 인스턴스 최대 1000개 이상 확장 가능한 구조로 설계 (Server가 DB 기반 라우팅 테이블로 물리 서버 위치를 관리)
- 자체 TCP 프로토콜로 파일 릴레이 시스템을 처음부터 구현 — 기존 운영 중인 실데이터를 건드리지 않도록 "베이스라인 보호" 로직 적용(서비스 시작 시점에 이미 존재하던 파일은 영구히 처리 대상에서 제외)
- Prometheus/Grafana 인프라 모니터링에 업무 지표를 통합 — textfile collector를 활용해 커스텀 지표를 추가하고, 기존 대시보드에 자연스럽게 편입
- 장애 알림 자동화 — 상태 폴링 로직에 전이 감지를 추가해 인스턴스 다운/복구 시 담당자 개인 메신저로 즉시 알림
- 동기 블로킹 API를 비동기로 리팩터링해서 UI 응답성 개선 (자세한 내용은 CASE\_STUDIES 참고)

자세한 기술적 문제해결 사례는 [CASE\\_STUDIES.md](#) 를, 사용 기술 스택은 [TECH\\_STACK.md](#) 를 참고해 주세요.

# 문제해결 사례 (Case Studies)

실제로 겪고 진단·해결한 기술적 문제 7가지를 정리했습니다. 코드는 실제 사내 코드를 그대로 옮긴 것이 아니라, 핵심 로직만 재구성한 예시입니다.

## 1. JS로 동적 생성한 폼에서만 발생하는 403 CSRF 오류

**상황:** 관리 화면의 "실행" 버튼을 누르면 항상 403 Forbidden이 발생. 그런데 서버가 렌더링한 일반

`<form>` 제출(예: 삭제 버튼)은 멀쩡히 동작했습니다.

**진단 과정:** 처음엔 curl로 재현을 시도하며 로그인 세션/쿠키 문제로 오판했습니다. 사용자가 "실제 브라우저에서도 같은 버튼이 안 된다"고 알려준 뒤에야, 실제 접속 로그(액세스 로그)의 403 응답을 근거로 진짜 버그임을 확정했습니다.

**원인:** Spring Security는 서버가 렌더링하는 `<form>` 에는 CSRF 히든 필드를 자동으로 넣어주지만, `document.createElement('form')` 으로 **JavaScript가 즉석에서 만든 폼에는 넣어주지 않습니다.** 버튼 하나만 JS로 폼을 동적 생성해서 제출하는 구조였는데, 이 폼에 CSRF 토큰이 아예 없었던 것입니다.

**해결:**

```
// 페이지에 이미 서버가 렌더링해둔 다른 폼에서 CSRF 토큰을 그대로 가져와 붙인다
var csrfInput = document.querySelector('input[name="_csrf"]');
if (csrfInput) {
  var hidden = document.createElement('input');
  hidden.type = 'hidden';
  hidden.name = '_csrf';
  hidden.value = csrfInput.value;
  form.appendChild(hidden);
}
```

**배운 점:** 재현이 안 될 때 "테스트 도구(curl) 문제"로 성급히 결론내리지 않고, 실제 사용자 환경의 증거(액세스 로그)를 근거로 재검증하는 습관이 중요하다는 걸 다시 확인한 사례.

## 2. 컨테이너 기본 시간대(UTC) 때문에 생긴 9시간 오차

**상황:** 테스트 파일을 생성하면 파일명에 타임스탬프가 찍히는데, 실제 생성 시각과 9시간이나 차이가 나는 현상 발견.

**진단 과정:** 호스트 서버 시각과 파일명 타임스탬프를 나란히 비교 → 정확히 9시간(UTC-KST 오프셋) 차이. 컨테이너에 직접 접속해 `date` 명령으로 확인한 결과, 컨테이너 내부 시스템 시간대가 UTC로 설정되어 있었음( `TZ=Asia/Seoul` 미설정).

**원인:** `LocalDateTime.now()` 는 JVM(=컨테이너 OS)의 기본 시간대를 따라가는데, 이 컨테이너의 베이스 이미지가 시간대를 명시적으로 지정하지 않아 UTC로 동작하고 있었음.

**해결:** 컨테이너 환경설정에 의존하지 않도록, 코드 레벨에서 시간대를 명시적으로 고정.

```
private static final ZoneId KST = ZoneId.of("Asia/Seoul");
...
String ts = LocalDateTime.now(KST).format(TS_FORMAT);
```

**배운 점:** 컨테이너 환경변수/설정에 의존하는 대신, 비즈니스 로직에서 필요한 시간대는 코드에서 명시적으로 고정하는 게 더 견고하다 — 나중에 다른 환경(다른 리전, 다른 베이스 이미지)에 배포되어도 동일하게 동작함이 보장됨.

---

### 3. Prometheus 집계 함수 선택 오류로 인한 "컨테이너 상태 2로 표시" 버그

---

**상황:** 모니터링 대시보드에서 정상 실행 중인 컨테이너 하나가 상태값 "1(UP)"이 아니라 "2"로 표시됨.

**진단 과정:** 해당 컨테이너의 원본 메트릭을 라벨까지 전부 조회 → 동일한 컨테이너인데 라벨 셋이 미묘하게 다른(예: 재시작 횟수 라벨 유무) 시계열이 **2개** 존재. `count by (name) (...)` 쿼리가 "같은 컨테이너인지"가 아니라 "라벨 조합이 몇 개인지"를 세고 있었던 것.

**원인:** `count()` 는 라벨 셋이 다르면 다른 시계열로 취급해 각각 카운트한다. 컨테이너가 재시작되면 cAdvisor가 일시적으로 신/구 라벨 조합을 동시에 노출하는 경우가 있어, 이런 상황에서 `count` 는 쉽게 부정확해진다.

**해결:** "존재 여부"만 필요한 경우엔 `count` 대신 `group` 을 사용하도록 변경.

```
# Before: 라벨 조합 개수를 세서 부정확
count by (name) (container_last_seen{...})

# After: 조합이 몇 개든 그룹당 값 1을 반환 → 존재 여부 체크에 안전
group by (name) (container_last_seen{...})
```

**배운 점:** Prometheus에서 "몇 개 있는지"가 아니라 "있는지 없는지"를 확인하려면 `count` 가 아니라 `group` 을 쓰는 게 라벨 카디널리티 변화에 안전하다는 걸 실전에서 체득.

---

### 4. 인프라 모니터링에 업무 지표를 자연스럽게 통합하기

---

**상황:** 기존 대시보드는 CPU/메모리/컨테이너 상태만 보여줬는데, "오늘 처리된 업무 파일 건수"도 같은 테이블에서 한눈에 보고 싶다는 요구사항.

**설계 및 구현:** 1. `node_exporter` 의 **textfile collector** 기능을 활성화(컴포즈 설정에 볼륨/커맨드 옵션 추가) 2. 각 인스턴스의 파일 카운트를 계산해 Prometheus 텍스트 포맷으로 내보내는 스크립트를 작성, cron으로 1분마다 실행 3. Grafana 패널에 새 쿼리를 추가하면서, 실제 존재하지 않는 인스턴스까지 유령 행으로 잡히는 부작용 발견 → PromQL의 `and on(label)` 연산자로 "실제 살아있는 컨테이너와 라벨이 겹치는 것만" 필터링해서 해결

```
sum by (name) (
  business_files_today{name=~"^instance.*"}
  and on(name) container_last_seen{name=~"^instance.*"}
)
```

**결과:** 인프라 지표와 업무 지표가 같은 테이블/같은 갱신주기(10초)로 한 화면에서 관리됨.

**배운 점:** 표준 모니터링 스택(Prometheus/Grafana)은 애초에 확장 가능하게 설계돼 있어서, "업무 지표는 별도 시스템이 필요하다"고 단정하기보다 기존 스택의 확장 포인트(textfile collector, PromQL 집합 연산)를 먼저 찾아보는 게 훨씬 적은 비용으로 문제를 해결할 수 있다는 걸 배움.

---

## 5. 재기동에도 살아남는 OAuth 토큰 생명주기 관리

---

**상황:** 인스턴스 상태 전이(다운/복구) 시 담당자 개인 메신저로 알림을 보내는 기능을 추가하게 됨. 해당 메신저 플랫폼의 알림 API는 6시간짜리 액세스 토큰 + 60일짜리 리프레시 토큰 구조.

**설계 고려사항:** - 액세스 토큰은 메모리에만 두고 6시간마다(만료 전에 5시간 주기로) 자동 갱신 - 리프레시 토큰은 갱신 응답에 새 값이 오면 **회전(rotation)**될 수 있음 → 매번 최신값을 파일에 원자적으로(temp 파일 write 후 rename) 저장해서 재기동해도 유지 - 앱 기동 시 저장된 리프레시 토큰으로 즉시 1회 갱신 해서 유효성 검증

```
@Scheduled(initialDelay = 5 * HOUR, fixedRate = 5 * HOUR)
public void scheduledRefresh() {
    if (isConfigured()) refresh();
}
```

**배운 점:** 외부 API의 토큰 생명주기(만료·회전)를 문서만 보고 넘어가지 않고, "재기동했을 때도 동작하는가", "토큰이 회전했을 때 놓치지 않는가"까지 설계에 반영하는 습관.

---

## 6. 블로킹 API를 비동기로 바꿔 UI 응답성 개선

---

**상황:** 테스트 파일을 N개, M초 간격으로 생성하는 기능에서, N과 M이 커질수록 브라우저가 (N-1)×M 초 만큼 그대로 멈춰있는 문제.

**원인:** 요청 스레드가 파일 생성 루프 안에서 매번 `Thread.sleep(M)` 을 호출하며 끝까지 블로킹된 뒤에야 응답을 반환하고 있었음 — HTTP 체인 전체(브라우저 → 중계서버 → 실행서버)가 동기로 연결돼 있어

가장 안쪽의 sleep이 그대로 브라우저까지 전파됨.

**해결:** 실행 요청은 유효성 검증만 동기로 처리하고 즉시 "접수됨" 응답을 반환, 실제 생성 작업은 백그라운드 스레드풀로 위임.

```
public Result simulate(...) {
    if (!isValid(...)) return Result.fail("...");
    executor.execute(() -> runInBackground(...));
    return Result.accepted("요청이 접수되었습니다. 백그라운드에서 처리 중입니다.");
}
```

프론트엔드도 "완료"가 아니라 "접수됨" 기준으로 문구를 수정하고, 실제 완료 여부는 별도 상태 조회 API로 확인하도록 UX를 조정.

**결과:** 응답시간이 (N-1)×M초에서 0.2초 내외로 단축. 대신 "요청이 접수됐다"와 "실제로 끝났다"는 다른 의미라는 걸 화면에서도 명확히 구분하도록 신경씀.

## 7. 자체 TCP 프로토콜 기반 파일 릴레이 시스템 (제로베이스 설계)

**상황:** 인스턴스별로 발신 폴더에 쌓이는 파일을 중앙 서버로, 중앙 서버에 쌓인 파일을 다시 각 인스턴스로 내려주는 양방향 릴레이가 필요. HTTP로도 가능했지만, 대용량 파일 스트리밍과 "새 파일이 있는지 계속 물어보는" pull 방식을 가볍게 구현하고 싶어서 raw TCP 소켓 기반 자체 프로토콜을 설계.

**프로토콜 설계:** - 연결 1개 = 요청 1건. 첫 1바이트로 opType(SEND/FETCH) 구분 -

DataInputStream / DataOutputStream 의 writeUTF / writeLong 으로 파일명·길이 프레이밍 - 수신측은 .part 임시파일에 먼저 쓰고 완료되면 원자적으로 rename (부분전송 방지) - FETCH는 요청 시점에 큐에 있는 가장 오래된 파일 1개를 스트리밍하고, 수신측의 ack를 받은 뒤에만 원본을 정리

**운영 데이터 보호 장치:** 발신측 서비스가 처음 뜰 때, 이미 그 폴더에 있던 기존 파일 목록을 스냅샷으로 떠서 "베이스라인"으로 저장하고, 이후 그 파일들은 영구히 처리 대상에서 제외했습니다. 실제 운영 중이던 폴더에 수년간 쌓인 기존 데이터를 새 시스템이 잘못 건드리지 않도록 하기 위한 안전장치였습니다.

**배운 점:** 표준 프로토콜(HTTP) 대신 자체 프로토콜을 선택할 땐 "왜 필요한가"를 분명히 하고, 운영 중인 실데이터에 새 자동화 로직을 엮을 때는 최소 권한/최소 영향 원칙(건드릴 대상을 명시적으로 좁히는 것)을 항상 먼저 고려해야 한다는 걸 체감.

# 기술 스택

## Backend

---

- **Java 8 / 21, Spring Boot 2.7**
- Spring MVC, Spring Security (폼 로그인, CSRF), Spring Data JPA
- Thymeleaf (서버사이드 렌더링 + 부분 AJAX 갱신)
- RESTful API 설계 (API 키 기반 서비스 간 인증)
- Raw TCP 소켓 프로그래밍 ( `Socket` , `DataInputStream` / `DataOutputStream` 기반 커스텀 바이너리 프로토콜)
- 스레드풀 기반 비동기 처리 ( `ExecutorService` , `@Scheduled` , `@Async` 패턴)
- Python (Flask) — 별도 사이드 프로젝트형 관리 도구

## 데이터베이스

---

- MariaDB, Spring Data JPA / Hibernate

## 인프라 / 운영

---

- Docker, Docker Compose (멀티 서비스 오케스트레이션)
- 컨테이너 레지스트리 운영(사실 레지스트리 push/pull)
- Linux 서버 운영 (CentOS 계열), systemd, cron, LVM/XFS 스토리지 이해
- 방화벽(firewalld) 설정, 포트/네트워크 트러블슈팅

## 모니터링 / 관측성

---

- Prometheus (PromQL 쿼리 작성, 집계 함수 트러블슈팅)
- Grafana (대시보드 설계, 패널/트랜스폼 API를 통한 프로그래매틱 대시보드 수정)
- cAdvisor, node\_exporter (+ textfile collector로 커스텀 지표 확장)

## 외부 연동

---

- OAuth2 스타일 토큰 발급/갱신 플로우 (엑세스/리프레시 토큰, 토큰 회전 대응)
- 외부 메신저 플랫폼 알림 API 연동



## 협업 / 형상관리

---

- Git / GitHub (커밋 단위 설계, 시크릿 유출 대응 — `commit --amend` + force push로 즉시 스크럽)
- 요구사항 검토 → 구현 → 실제 서버 배포 → end-to-end 검증까지 전 과정을 반복적으로 수행하는 워크플로우에 익숙

## 문제해결 접근 방식

---

- 로그/메트릭 기반 근거 있는 디버깅 (추측보다 실제 증거 우선)
- 재현 어려운 버그는 실제 사용자 환경(브라우저, 접속 로그)까지 파고들어 확인
- 운영 중인 시스템에 새 기능을 엮을 때 기존 데이터/트래픽에 대한 영향도를 먼저 검토